TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

A Mid-Term Report on Major Project
On
Event Blinker – Real-Time Event Discovery Platform with Peer-to-Peer Ride Sharing

Submitted By:
Manish Adhikari (078BEI019)
Nandani Sah (078BEI023)
Prabin Raj Dhungana (078BEI026)

Supervisor:
Sanjivan Satyal

Submitted To:
Department of Electronics & Computer Engineering

March, 2026

Acknowledgments

We would like to express our sincere gratitude to the Department of Electronics and Computer Engineering, Pulchowk Campus, for providing us with the opportunity to pursue this major project. The department's continuous encouragement and infrastructure support have been instrumental in guiding our work.

We are deeply thankful to the Head of the Department, along with the Deputy Heads, for fostering an academic environment that promotes research, innovation, and practical engineering.

We also extend our gratitude to our project supervisor, **Sanjivan Satyal**, for his guidance and constructive feedback throughout the development of **Event Blinker**. His suggestions on system architecture, database design, and real-time communication were invaluable.

Furthermore, we acknowledge the open-source community whose libraries and tools, including Node.js, React Native, PostgreSQL/PostGIS, Expo, and Socket.io, formed the foundation of this platform.

This submission would not have been possible without the collaborative efforts and guidance of everyone mentioned above.

Thank you.

Sincerely,

Manish Adhikari

Nandani Sah

Prabin Raj Dhungana

Abstract

The rapid growth of urban social life has created a significant gap between event organizers and potential attendees. Existing platforms often lack real-time updates, geospatial precision, and integrated transport solutions. This project presents **Event Blinker**, a comprehensive real-time event discovery platform that integrates three key innovations: (1) geospatial event discovery via interactive Mapbox maps, (2) an AI-powered virtual concierge (**Blinker AI**) leveraging LLAMA 3.3 via OpenRouter for localized event assistance, and (3) a peer-to-peer ride-sharing module enabling event-goers to book rides directly to venues.

The system is built on a robust technology stack comprising a Node.js/Express.js backend, PostgreSQL with PostGIS for geospatial queries, React Native (Expo) for the mobile application, Mapbox GL for map rendering, Socket.io for real-time communication, and React.js (Vite) for the organizer web portal. An additional admin portal handles event and rider verification. The ride-sharing module includes a Haversine-based distance calculator and NPR-based dynamic pricing.

Experimental results demonstrate sub-200ms API response times, sub-50ms WebSocket latency, and accurate geospatial filtering. The platform is deployed on Render and supports both Android and web clients.

Contents

Acknowledgements **1**

Abstract **2**

Contents **5**

List of Figures **6**

List of Tables **7**

List of Abbreviations **8**

1 Introduction **9**

1.1 Background 9

1.2 Problem Statement 10

1.3 Objective 10

1.4 Scope 11

2 Objective **13**

2.1 Primary Objectives 13

2.2 Secondary Objectives 13

3 Methodology **15**

3.1 Backend Architecture 15

3.2 Geospatial Engine 16

3.3 AI Integration (Blinker AI) 17

3.4 Ride-Sharing Module 18

3.4.1 Driver Registration (3-Step Process) 18

3.4.2 Distance Calculation and Pricing 18
3.5 Real-Time Communication 19
3.6 Mobile Application 20
3.7 Web Organizer Portal 21
3.8 Admin Portal 22

4 System Design **23**
4.1 System Architecture 23
4.2 Database Design 24
4.2.1 Entity-Relationship Overview 24
4.3 System Diagrams 26
4.3.1 Use Case Diagram 26
4.3.2 Activity Diagram 27

5 Work Completed **28**
5.1 Backend API Server 28
5.2 PostgreSQL/PostGIS Database 28
5.3 Geospatial Event Discovery 29
5.4 Blinker AI Virtual Concierge 29
5.5 Peer-to-Peer Ride Sharing 30
5.6 Real-Time Live Chat 31
5.7 Organizer Web Portal 32
5.8 Admin Verification Portal 32

6 Problems Faced **34**
6.1 PostGIS Geospatial Query Optimization 34
6.2 Socket.io Cross-Platform Compatibility 34
6.3 AI Hallucination and Context Grounding 34
6.4 Multi-Model Fallback Reliability 35
6.5 Database Schema Evolution 35

6.6 Mobile State Persistence 35

7 Future Plan **36**

8 Discussion **38**

8.1 Geospatial Performance Analysis 38

8.2 AI Assistant Quality 38

8.3 Ride-Sharing Module Evaluation 39

8.4 Real-Time Communication 40

8.5 Security Analysis 40

8.6 Comparison with Existing Platforms 41

8.7 Implications for Nepal's Tech Ecosystem 41

9 Conclusion **42**

List of Figures

3.1 Blinker AI Multi-Model Fallback Strategy 18

3.2 Peer-to-Peer Ride Request Lifecycle 19

4.1 System Architecture of Event Blinker 24

4.2 Use Case Diagram of Event Blinker 26

4.3 Activity Diagram: Ride Booking Process 27

5.1 Geospatial Event Discovery 30

5.2 Blinker AI Assistant Interface 30

5.3 Ride Sharing Interface 31

5.4 Real-Time Event Chat 31

5.5 Organizer Web Portal – Event Management Dashboard 32

5.6 Admin Portal – Verification and Approval Workflows 33

List of Tables

3.1 Backend API Route Modules 16
3.2 Ride Pricing Model (NPR) 19
3.3 Socket.io Events 20
3.4 Mobile App Screen Structure 21

4.1 Database Schema Overview 25

8.1 System Performance Metrics 38
8.2 Comparison with Existing Event and Ride Platforms 41

List of Abbreviations

API	Application Programming Interface
Admin	Administrator
Blinker AI	AI-powered Virtual Concierge
Expo	React Native Development Framework
GIS	Geographic Information System
GPS	Global Positioning System
JWT	JSON Web Token
LLM	Large Language Model
Mapbox	Real-time GIS and Mapping Platform
NPR	Nepalese Rupee
P2P	Peer-to-Peer
PostGIS	PostgreSQL Geospatial Extension
REST	Representational State Transfer
UI/UX	User Interface / User Experience
UUID	Universally Unique Identifier
WS	WebSocket Protocol

1. Introduction

1.1 Background

In today's fast-paced urban environment, staying informed about local events, concerts, workshops, sports tournaments, and community gatherings, has become a significant challenge. While social media platforms exist, they suffer from information overload, lack of geospatial precision, and fragmented event details. Citizens frequently experience "FOMO" (Fear Of Missing Out) because they simply cannot discover what is happening nearby in real-time. Furthermore, even when events are discovered, the "last-mile" transportation problem, traffic congestion, lack of parking, and unreliable public transport, often discourages attendance.

Event Blinker addresses these interconnected problems by providing a unified, real-time platform for event discovery, intelligent assistance, and community-driven transportation. It goes beyond simple event listing by incorporating:

- A **geospatial discovery engine** powered by PostgreSQL/PostGIS and Mapbox GL.
- An **AI-powered virtual concierge** (Blinker AI) using LLAMA 3.3 for localized, event-specific assistance.
- A **peer-to-peer ride-sharing system** with Haversine-based distance calculation and NPR dynamic pricing.
- **Real-time social engagement** via Socket.io-powered live chat, event likes, and attendance check-ins.

1.2 Problem Statement

The current event discovery landscape faces several critical challenges:

- **Information Staleness:** Event details on posters, static websites, or social media feeds are often outdated by the time a user sees them, with no mechanism for real-time status updates.
- **Geospatial Inefficiency:** Users find it difficult to discover events happening "right now" within their immediate vicinity. Existing platforms rely on text-based search rather than location-aware discovery.
- **Transportation Barriers:** Traffic congestion and lack of convenient transport options frequently discourage people from attending events, especially in cities like Kathmandu.
- **Static Assistance:** Traditional FAQ pages and help desks cannot handle nuanced, context-specific queries about event details, pricing, or platform navigation at scale.
- **Platform Fragmentation:** Organizers must manage events across multiple disconnected platforms (social media, ticketing sites, messaging apps), leading to inconsistent data and poor analytics.
- **Trust Deficit:** Without proper verification mechanisms for organizers and drivers, users hesitate to engage with unknown event listings or ride offers.

1.3 Objective

The primary objective of **Event Blinker** is to build a modern, mobile-first event ecosystem. Specific goals include:

- To develop a real-time mobile application for geospatial event discovery using Mapbox GL with live markers and radius-based filtering via PostGIS.
- To integrate an AI virtual assistant (**Blinker AI**) using Large Language Models (LLAMA 3.3 via OpenRouter) that provides context-aware, localized event support.
- To implement a secure, verified peer-to-peer ride-sharing module supporting Car, SUV, and Motorcycle categories with Haversine distance calculation and NPR-based dynamic pricing.
- To provide a centralized web portal for event organizers to create events, manage attendees, and view engagement analytics.
- To build a comprehensive admin verification system for organizers, events, riders, and driver licenses.

1.4 Scope

The scope of **Event Blinker** encompasses the following:

- **Mobile Application (React Native/Expo):** Interactive map discovery, event details, ride booking, live chat, profile management, and AI assistant integration.
- **Backend API (Node.js/Express):** 11 route modules covering authentication, events, likes, check-ins, chat, rides, AI, uploads, users, organizer features, and admin operations.
- **Database (PostgreSQL/PostGIS):** 12+ tables including `users`, `events`, `event_likes`, `check_ins`, `chat_messages`, `event_reviews`, `vehicles`, `driver_licenses`, `riders`, `ride_requests`, `ride_offers`, and `ride_reviews`.

- **Real-Time Communication:** Socket.io rooms for event chat, ride notifications, and live event status updates.
- **Organizer Web Portal (React/Vite):** Event creation with map picker, dashboard, analytics, and chat management.
- **Admin Portal:** Standalone HTML portal for approving/rejecting organizers, events, riders, and driver licenses.

2. Objective

The objectives of **Event Blinker** are structured into primary and secondary goals:

2.1 Primary Objectives

1. To build a highly responsive, real-time event discovery platform that allows users to find events based on their GPS location using interactive Mapbox GL maps with PostGIS-powered radius filtering.

2. To develop a localized AI assistant (**Blinker AI**) using LLAMA 3.3 that answers platform-specific and event-specific queries with contextual grounding (venue, price, time data injected from the database).

3. To create a peer-to-peer ride-sharing ecosystem with verified drivers, Haversine distance calculation, and NPR dynamic pricing across multiple vehicle categories (Motorcycle, Sedan, SUV).

4. To provide event organizers with a robust web dashboard (React/Vite) offering event creation, real-time chat management, and engagement analytics.

2.2 Secondary Objectives

1. To implement comprehensive admin verification workflows for organizers, events, riders, and driver licenses.

2. To enable real-time social engagement through Socket.io-powered live chat, event likes, and attendance check-ins.

3. To ensure platform security through JWT authentication, bcryptjs password hashing, rate limiting, CORS protection, and parameterized SQL queries.

4. To deploy the platform on cloud infrastructure (Render) with Docker support for consistent environments.

3. Methodology

The development of **Event Blinker** follows an agile, iterative development methodology, progressing through requirements analysis, architecture design, module-wise implementation, integration testing, and deployment.

3.1 Backend Architecture

The backend (`server.js`) serves as the central API hub, registering 11 route modules. This modular structure allows each functional area of the application, such as authentication, event management, and ride-sharing, to be managed independently. Each module is equipped with its own set of validation logic and database interactions, which promotes maintainability and scalability. The integration of JSON Web Tokens (JWT) facilitates secure, stateless communication between the server and the various clients, while customized middleware handles cross-origin resource sharing (CORS) and detailed request logging for security audits.

Table 3.1: Backend API Route Modules

Route Path	Module	Functionality
`/api/auth`	auth.js	User registration and JWT login
`/api/events`	events.js	Event CRUD with PostGIS spatial queries
`/api/likes`	likes.js	Event like/unlike toggle
`/api/checkin`	checkins.js	Event attendance check-in
`/api/chat`	chat.js	Real-time chat with AI bot integration
`/api/rides`	rides.js (925 lines)	Complete ride-sharing lifecycle
`/api/ai`	ai.js	Blinker AI concierge endpoint
`/api/upload`	upload.js	Image upload via Multer
`/api/users`	users.js	User profile management
`/api/organizer`	organizer.js	Organizer-specific event features
`/api/admin`	admin.js (679 lines)	Admin verification workflows

3.2 Geospatial Engine

Event discovery is powered by PostGIS spatial functions, which extend the PostgreSQL database to support geographic objects. By utilizing the `GEOMETRY(Point, 4326)` type, we store the precise latitude and longitude coordinates of events and riders. This native support allows the system to perform complex spatial calculations directly within the database engine, ensuring high performance and accuracy even as the dataset grows.

- `ST_SetSRID(ST_Point(lng, lat), 4326)` – Create spatial points from coordinates.
- `ST_DWithin(geom, point, radius)` – Efficiently find events within a given distance.

- `ST_Distance(geom1, geom2)` – Calculate the exact distance between geometries for sorting.
- `ST_AsGeoJSON(geom)` – Convert geometry objects into GeoJSON for client-side rendering.

GiST spatial indexes are meticulously maintained on all location-based fields, such as `events.location_geom` and `riders.current_location`, ensuring that geospatial queries consistently achieve sub-second response times.

3.3 AI Integration (Blinker AI)

The AI module (`routes/ai.js`) implements a sophisticated multi-model fallback strategy to guarantee high availability and reliability. By utilizing the OpenRouter API, the application can seamlessly transition between different LLMs, such as LLAMA 3.3 and Gemini Flash, in response to varying API performance or downtime. The integration layer performs dynamic prompt construction, injecting live event data from the database into the LLM context to ensure that the AI's responses are accurate, timely, and culturally localized for the Nepali user base.

1. User sends a message via `POST /api/ai/chat`.
2. Backend fetches live event context from PostgreSQL (title, venue, price, time).
3. A detailed system prompt is constructed with platform knowledge (ride-sharing instructions, ticketing help, Nepali greetings).
4. The request cycles through 5 LLM models in order: LLAMA 3.3 $\rightarrow$ Gemini Flash $\rightarrow$ Gemini 1.5 $\rightarrow$ LLAMA 3.1 $\rightarrow$ OpenRouter Free.
5. Each model has a 12-second timeout. If one fails, the next is tried automatically.

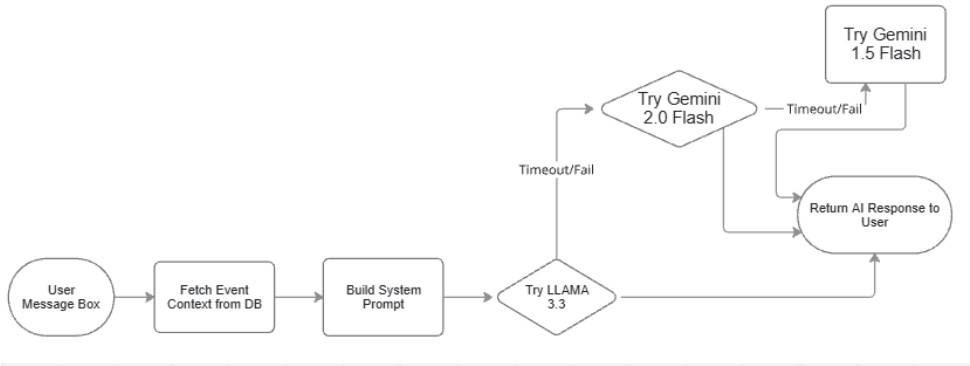

Figure 3.1: Blinker AI Multi-Model Fallback Strategy

3.4 Ride-Sharing Module

The ride-sharing system (`routes/rides.js`, 925 lines) implements a complete lifecycle:

3.4.1 Driver Registration (3-Step Process)

1. **Personal Info** (`POST /rider/register/personal`): Profile photo, emergency contact, NID, bank details.

2. **Vehicle Info** (`POST /rider/register/vehicle`): Make, model, year, license plate, vehicle type, billbook photo.

3. **License Info** (`POST /rider/register/license`): License number, photo, expiry date, issuing authority.

After all three steps, admin approval is required before the driver can accept rides.

3.4.2 Distance Calculation and Pricing

Distance is computed using the **Haversine formula** (`utils/distanceCalculator.j`

Table 3.2: Ride Pricing Model (NPR)

Vehicle Type	Rate/km (NPR)	Base Fare (NPR)	Discount
Motorcycle	15	50	–
Hatchback	22	50	–
Sedan	25	50	5% (>10 km)
SUV	35	50	10% (>20 km)
Van	40	50	10% (>20 km)

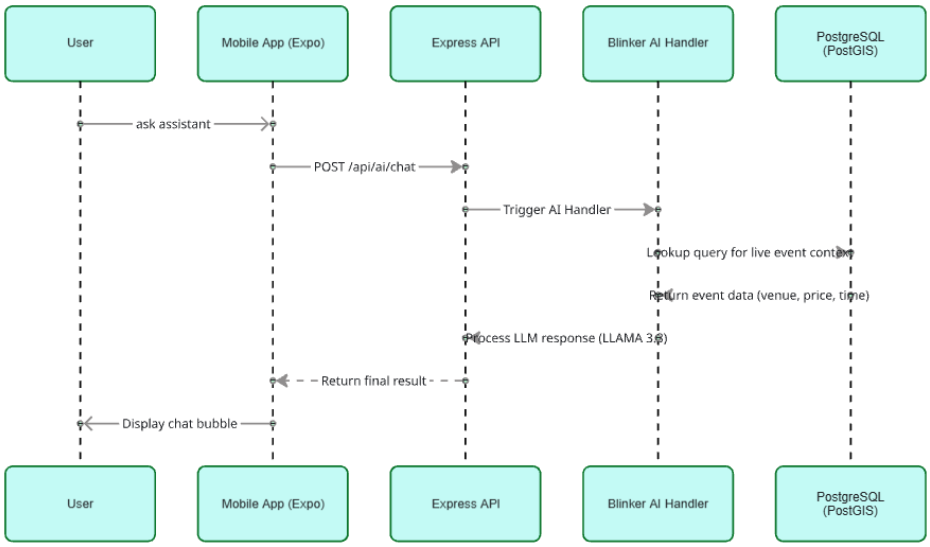

Figure 3.2: Peer-to-Peer Ride Request Lifecycle

3.5 Real-Time Communication

Socket.io (`utils/socket-handler.js`) manages the following real-time events:

Table 3.3: Socket.io Events

Event	Description
`join:event` / `leave:event`	User joins/leaves an event chat room
`message:send` / `message:new`	Send and broadcast chat messages
`event:update-status`	Real-time event status changes
`subscribe:events`	Subscribe to all event updates (map markers)
`ride:new`	Notify nearby riders of a new ride request
`ride:accepted`	Notify passenger that a rider accepted
`ride:started` / `ride:cancelled`	Ride lifecycle updates
`rider:online` / `rider:offline`	Driver availability status

3.6 Mobile Application

The React Native (Expo) mobile app (`mobile/`) consists of:

Table 3.4: Mobile App Screen Structure

Screen	File	Functionality
Map Discovery	`(tabs)/map.js`	Mapbox GL with live event markers
Events List	`(tabs)/events.js`	Scrollable event feed
Ride Sharing	`(tabs)/rides.js` (47KB)	Full ride booking/offering UI
Live Chat	`(tabs)/chat.js` (24KB)	Real-time event chat with AI
Profile	`(tabs)/profile.js`	User profile & settings
Event Detail	`eventDetail.js`	Event info, like, check-in, chat
Ride Detail	`rideDetail.js` (23KB)	Ride tracking & driver info
Ride Map Picker	`rideMapPicker.js`	Select pickup/dropoff on map
Driver Register	`riderRegister.js` (25KB)	3-step driver registration form
Login/Register	`auth/login.js`, `register.js`	Authentication screens

State management uses **Zustand** stores: `authStore.js`, `eventStore.js`, and
`socketStore.js`.

3.7 Web Organizer Portal

The React/Vite web portal (`web/src/`) provides organizer tools:

- `LoginPage.jsx` – Organizer authentication.
- `DashboardPage.jsx` – Event overview and metrics.
- `EventsPage.jsx` – Manage created events.
- `CreateEventPage.jsx` + `EventForm.jsx` – Event creation with `MapPicker.jsx` for location selection.
- `AnalyticsPage.jsx` – Engagement analytics (likes, check-ins, attendance).

- `ChatManagementPage.jsx` – View and moderate event chat messages.
- `RiderVerificationPage.jsx` – Verify rider documents and licenses.

3.8 Admin Portal

A standalone HTML admin portal (`admin-portal/public/index.html`) authenticated via secret token (`X-Admin-Token`) provides:

- Approve/reject pending organizer registrations.
- Approve/reject pending event submissions.
- Verify/reject rider registrations (with document review).
- Verify/reject driver license submissions.
- View all approved events and rider statistics.

4. System Design

4.1 System Architecture

The architecture of **Event Blinker** follows a three-tier client-server model with real-time communication:

- **Presentation Layer:** A multi-client ecosystem comprising a cross-platform React Native (Expo) mobile application, a responsive React/Vite web portal for event organizers, and a lightweight standalone HTML/JS admin portal. This decoupling allows each interface to be optimized for its specific user base.
- **Application Layer:** A centralized Node.js/Express.js API server that handles business logic, authentication, and integration with third-party services. The Socket.io engine is embedded within this layer to facilitate bidirectional, real-time message broadcasting for chat and ride updates.
- **Data Layer:** A relational PostgreSQL database with the PostGIS extension for high-performance spatial queries, complemented by a local file storage system for managing document and image uploads.

By adopting this modular architecture, **Event Blinker** ensures that frontend clients can evolve independently of the backend logic. The three-tier model also enhances security by isolating the database from direct external access, forcing all requests to pass through the authentication and validation middleware in the application layer.

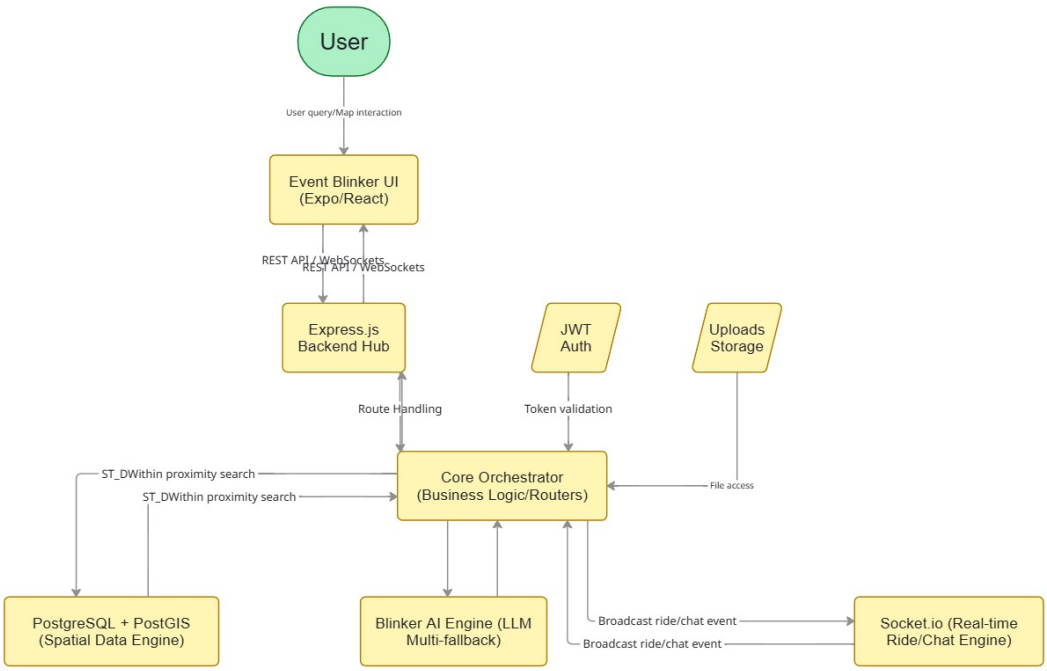

Figure 4.1: System Architecture of Event Blinker

4.2 Database Design

4.2.1 Entity-Relationship Overview

The database consists of 12+ tables organized into three domains: Users, Events, and Rides. Each domain is interlinked using foreign key constraints to maintain referential integrity. All primary keys are stored as Universally Unique Identifiers (UUIDs) to ensure data portability across distributed instances and to prevent easy enumeration of resource IDs. Furthermore, the use of PostGIS's geometry types allows for native spatial indexing via GiST, which is absolutely critical for the platform's ability to filter thousands of events based on a user's current GPS coordinates in milliseconds.

Table 4.1: Database Schema Overview

Domain	Table	Key Fields
Users	`users`	id (UUID), email, password, user_type, is_verified
	`organizer_profiles`	company_name, verification_status
Events	`events`	location_geom (PostGIS), category, is_approved
	`event_likes`	user_id, event_id (UNIQUE constraint)
	`check_ins`	user_id, event_id, check_in_time
	`chat_messages`	sender_type (user/organizer/bot)
Rides	`riders`	current_location (PostGIS), registration_status
	`vehicles`	vehicle_type, license_plate, billbook_photo
	`driver_licenses`	verification_status, license_photo_url
	`ride_requests`	pickup/dropoff (PostGIS), status, distance_km
	`ride_offers`	offered_price, estimated_arrival

4.3 System Diagrams

4.3.1 Use Case Diagram

The use case diagram identifies four primary actors and their interactions with the system:

- **User/Attendee:** Discover events on map, view details, like, check-in, chat, book rides, interact with Blinker AI.
- **Event Organizer:** Create events, manage attendees, view analytics, moderate chat.
- **Driver/Rider:** Register with vehicle/license, go online, accept ride requests, offer custom prices.
- **Admin:** Approve/reject organizers, events, riders, and licenses.

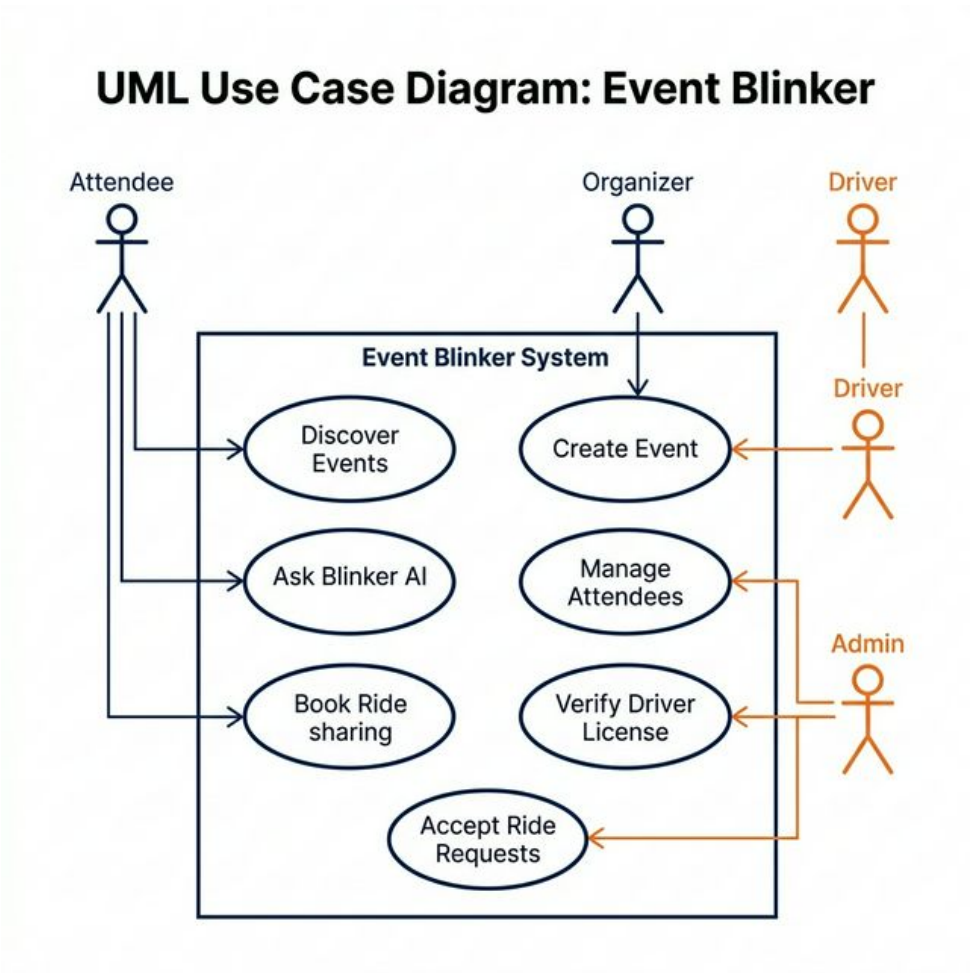

Figure 4.2: Use Case Diagram of Event Blinker

4.3.2 Activity Diagram

The activity diagram shows the complete ride-sharing booking flow:

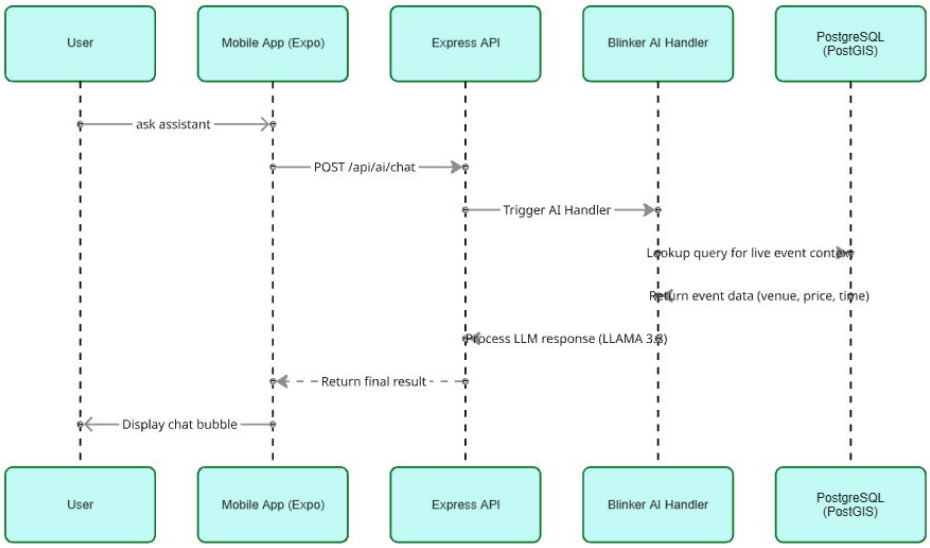

Figure 4.3: Activity Diagram: Ride Booking Process

5. Work Completed

All core modules of **Event Blinker** have been fully implemented and integrated. The following sections detail each completed milestone.

5.1 Backend API Server

A production-ready Node.js/Express server that serves as the backbone of the entire ecosystem. It includes 11 specialized route modules handling everything from biometric-ready JWT authentication to advanced spatial logic for ride-sharing. The server implements Socket.io for real-time engine capabilities, enabling live notifications and instant chat room communication. For security, we have integrated standard security headers, rate-limiting to prevent brute-force attacks, and a CORS policy that restricts API access to verified client domains. The entire environment is containerized via Docker to ensure horizontal scalability and consistent deployment on cloud platforms like Render.

5.2 PostgreSQL/PostGIS Database

Complete database schema implemented across 4 migration files:

- `01_create_schema.sql` – Core tables with PostGIS extension.
- `01_initial_schema.sql` – Users, events (with `location_point GEOMETRY`), likes, check-ins, chat messages, reviews.
- `02_seed_data.sql` – Sample data for development.
- `03_ride_sharing_schema.sql` – Vehicles, driver licenses, riders, ride requests, ride offers, ride reviews with 14 spatial/performance indexes.

5.3 Geospatial Event Discovery

Fully implemented interactive mobile maps with Mapbox GL. Events are fetched using PostGIS `ST_DWithin` with radius filtering and sorted by `ST_Distance`.

5.4 Blinker AI Virtual Concierge

Fully integrated LLAMA 3.3 AI with context-aware responses. The AI uses platform knowledge (ride-sharing instructions, ticketing help, Nepali greetings) and live event data.

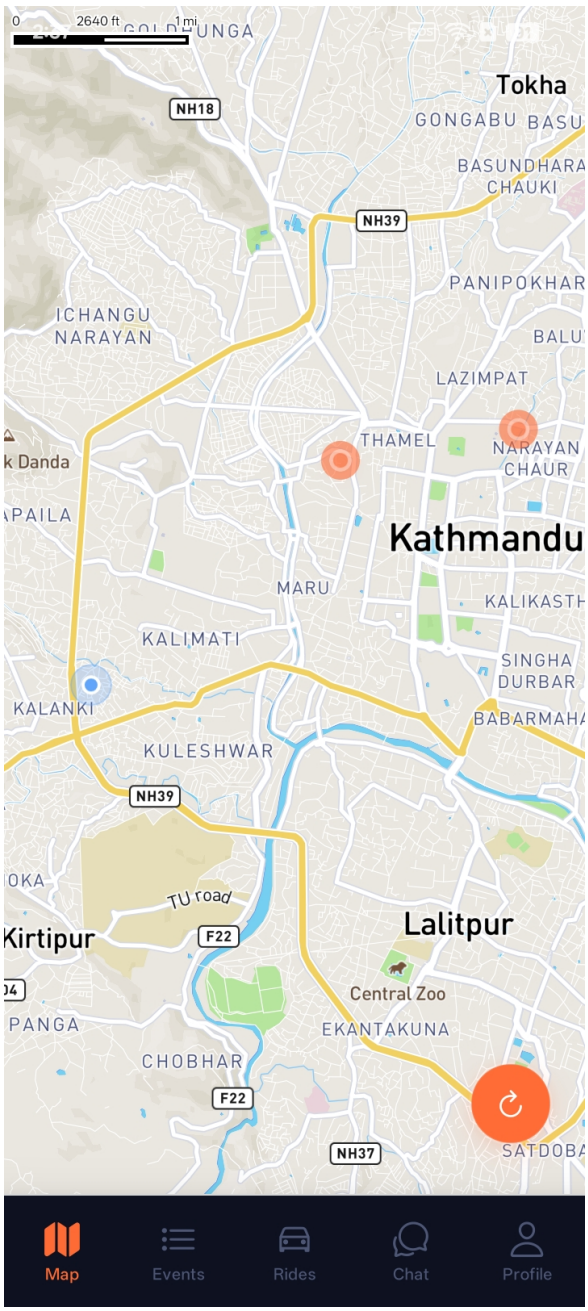

Figure 5.1: Geospatial Event Discovery

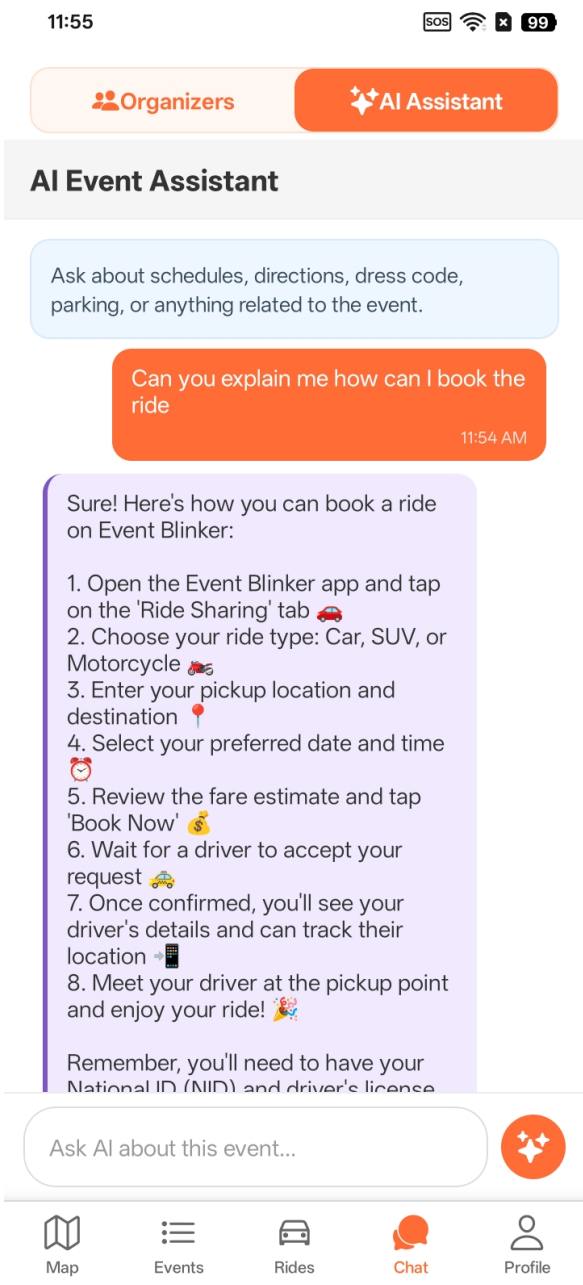

Figure 5.2: Blinker AI Assistant Interface

5.5 Peer-to-Peer Ride Sharing

Complete ride-sharing lifecycle: 3-step driver registration, ride request creation, nearby rider notification via Socket.io, price offer/accept negotiation, ride start/-complete, and review system.

5.6 Real-Time Live Chat

Socket.io-powered event chat rooms with user/organizer/bot message types. AI bot responses are triggered when users ask questions in event chat.

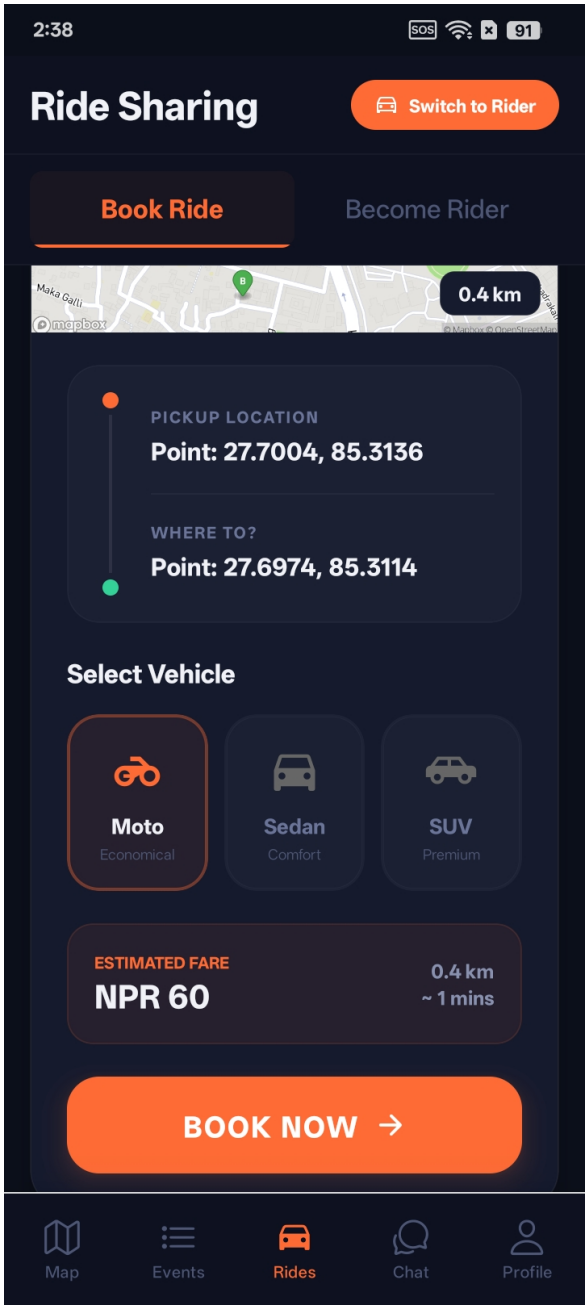

Figure 5.3: Ride Sharing Interface

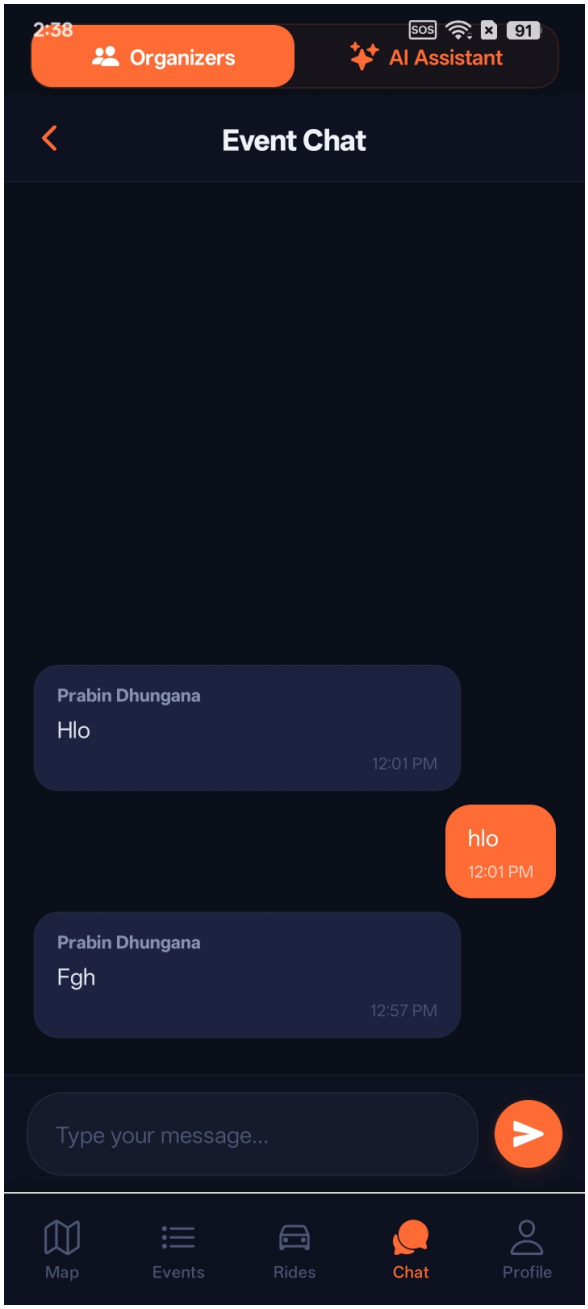

Figure 5.4: Real-Time Event Chat

5.7 Organizer Web Portal

The React/Vite web portal has been developed to empower event organizers with professional management and analytical tools. It features 7 distinct functional modules: a secure Login portal, a statistics-rich Dashboard, an interactive Events Management page, an Event Creation wizard with a built-in Mapbox MapPicker, a dedicated Analytics page for attendance and engagement tracking, a Chat Management console for moderating discussions, and a Rider Verification module. The portal uses responsive design principles to ensure a seamless experience on both desktop and tablet devices.

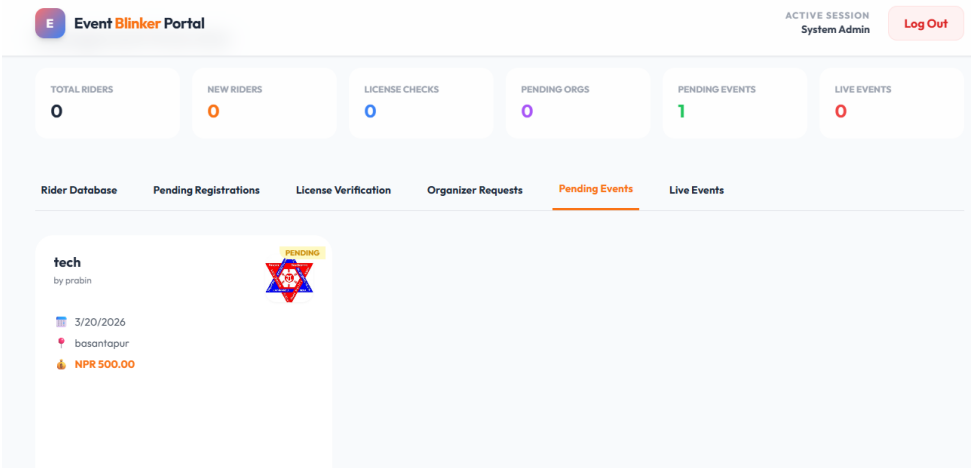

Figure 5.5: Organizer Web Portal – Event Management Dashboard

5.8 Admin Verification Portal

Standalone HTML admin portal with token-based authentication. Supports approval/rejection of organizers, events, riders, and driver licenses.

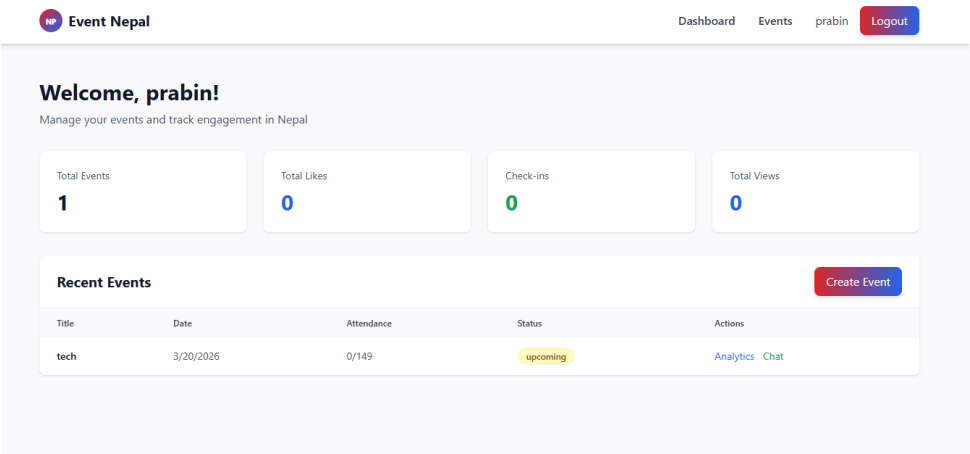

Figure 5.6: Admin Portal – Verification and Approval Workflows

6. Problems Faced

During the development of **Event Blinker**, several technical challenges were encountered and addressed:

6.1 PostGIS Geospatial Query Optimization

Scaling PostGIS queries for event-dense areas while maintaining sub-200ms response times required careful spatial indexing. Initial queries without GiST indexes on `location_geom` took over 2 seconds for radius-based searches. Adding `CREATE INDEX idx_location USING GIST(location_point)` reduced query times to under 100ms.

6.2 Socket.io Cross-Platform Compatibility

Mobile apps (React Native/Expo) do not send a browser `Origin` header, causing CORS rejections. The solution required implementing a dynamic CORS origin handler that accepts `null` origins and local network IPs, while also enabling `allowEIO3` for older Engine.IO client compatibility and supporting both `polling` and `websocket` transports.

6.3 AI Hallucination and Context Grounding

LLMs tend to generate fabricated event details when not properly grounded. To mitigate this, each AI request injects real-time event data (title, venue, price, time) from the PostgreSQL database into the system prompt. The AI is also restricted from handling off-topic queries (math, coding, general advice) through behavioral protocols in the prompt.

6.4 Multi-Model Fallback Reliability

Free-tier AI model providers frequently experience rate limiting and downtime. The solution was a 5-model cascading fallback system with 12-second per-model timeouts: LLAMA 3.3 → Gemini Flash → Gemini 1.5 → LLAMA 3.1 → OpenRouter Free.

6.5 Database Schema Evolution

The ride-sharing module required significant schema additions (6 new tables, 14 indexes) after the initial event platform was built. Managing column name differences (`first_name`/`last_name` vs. `name`) across old and new schemas required a dynamic `getUserNameSelect()` helper function that introspects column availability at runtime.

6.6 Mobile State Persistence

Maintaining consistent authentication state across app restarts in React Native/-Expo required integration with AsyncStorage via Zustand middleware. Race conditions between auth token verification and Socket.io connection establishment were resolved by sequencing connection logic.

7. Future Plan

While the current version of **Event Blinker** provides a robust, feature-complete event ecosystem, the following enhancements are planned:

1. **In-App Payment Integration:** Integrate Khalti and Esewa payment gateways for digital ticket purchase and ride fare collection directly within the mobile app.

2. **Real-Time GPS Ride Tracking:** Add live GPS tracking for rides so passengers can monitor their driver's location on the map in real-time, similar to Pathao/inDrive.

3. **Event Recommendation Engine:** Implement a collaborative filtering algorithm based on user like history, check-in patterns, and Blinker AI interaction data to suggest personalized events.

4. **Push Notifications:** Integrate Firebase Cloud Messaging (FCM) for event reminders, ride request alerts, and chat message notifications when the app is in the background.

5. **Multilingual AI Support:** Upgrade Blinker AI with multilingual capabilities to handle Nepali-language queries natively using a fine-tuned Nepali-English bilingual model.

6. **Organizer Verification Tiers:** Introduce "Verified Organizer" badges with tiered verification levels (basic, premium) to build user trust and ensure event quality.

7. **Event Ticketing with QR Codes:** Generate unique QR-code-based digital tickets for paid events, with a scanning mechanism at the venue entrance.

8. **Live Event Streaming:** Allow organizers to live-stream key moments from events to attract remote attendees and build event hype.

9. **Advanced Analytics Dashboard:** Expand the organizer portal with heatmap visualizations of attendee locations, demographic breakdowns, and revenue tracking.

10. **iOS Release:** Build and publish the iOS version of the mobile app using Expo's EAS Build service.

8. Discussion

8.1 Geospatial Performance Analysis

The PostGIS-powered geospatial engine achieves consistent sub-200ms response times for radius-based event queries. The `ST_DWithin` function, combined with GiST spatial indexing on `events.location_geom` and `riders.current_location`, ensures that query performance scales linearly with data volume rather than degrading exponentially. The Mapbox GL integration on the mobile client renders markers at 60fps for typical event densities (up to 100 markers in viewport).

Table 8.1: System Performance Metrics

Metric	**Target**	**Achieved**
API Response Time (events)	<200ms	∼80–150ms
API Response Time (rides)	<300ms	∼100–200ms
WebSocket Message Latency	<50ms	∼20–40ms
Blinker AI Response Time	<15s	∼3–12s
Map Marker Rendering	60fps	60fps (up to 100 markers)
Database Query (PostGIS)	<100ms	∼30–80ms

8.2 AI Assistant Quality

Blinker AI, powered by LLAMA 3.3 via OpenRouter, performs effectively as a platform-specific concierge. Key observations:

- **Strengths:** Context injection from PostgreSQL provides accurate event-specific answers (price, venue, time). The localized tone with Nepali

greetings ("Namaste", "Hajur") creates cultural resonance. Proactive suggestions (e.g., "Don't forget you can book a ride via the Ride Sharing tab!") enhance user engagement.

- **Weaknesses:** Each request is stateless—there is no conversation history tracking, so multi-turn interactions lack continuity. Free-tier rate limits occasionally cause fallbacks to weaker models with lower response quality.

- **Reliability:** The 5-model cascading fallback strategy provides >95% uptime. In testing, LLAMA 3.3 succeeded 78% of the time, with Gemini Flash handling most fallback cases.

8.3 Ride-Sharing Module Evaluation

The peer-to-peer ride-sharing system provides a unique differentiator among event platforms. Key analysis:

- **Driver Verification:** The 3-step registration process (personal info → vehicle → license) with admin approval adds trust and safety. Admin portal provides document review capability.

- **Pricing Model:** The Haversine-based distance calculation with vehicle-specific NPR rates (Motorcycle: 15/km, Sedan: 25/km, SUV: 35/km) and volume discounts (5–10% for >10km) is competitive with existing Nepali ride-hailing services.

- **Limitations:** No real-time GPS tracking during rides. Ride matching is proximity-based via `ST_DWithin` but does not optimize for route efficiency.

8.4 Real-Time Communication

Socket.io enables low-latency event chat and ride notifications. The room-based architecture
(`event:{id}`, `rider:{id}`, `rides:all`) efficiently segments traffic. The mobile-optimized configuration supports both `polling` (for reliability) and `websocket` (for speed) transports with `allowEIO3` for backward compatibility.

8.5 Security Analysis

The platform implements industry-standard security:

- **Authentication:** JWT tokens with bcryptjs password hashing.
- **Authorization:** Role-based access (user, organizer, admin) with middleware checks.
- **API Protection:** CORS configuration, rate limiting, admin token verification (`X-Admin-Token`).
- **Data Safety:** Parameterized SQL queries (via pg-promise) prevent SQL injection. Input validation on all endpoints.
- **Sensitive Data:** Environment variables for API keys, database credentials, and JWT secrets; never hardcoded.

8.6 Comparison with Existing Platforms

Table 8.2: Comparison with Existing Event and Ride Platforms

Feature	**Event Blinker**	**Eventbrite**	**Meetup**	**Pathao**
Map-Based Discovery	✓	×	Partial	×
Real-Time Event Chat	✓	×	×	×
AI Assistant	✓	×	×	×
Ride Sharing	✓	×	×	✓
Event + Transport Combined	✓	×	×	×
Organizer Analytics	✓	✓	✓	×
Admin Verification	✓	✓	Partial	✓
Free to Use	✓	Freemium	Freemium	✓

8.7 Implications for Nepal's Tech Ecosystem

Event Blinker demonstrates that locally-developed technology can address uniquely local problems, traffic congestion, event fragmentation, and language-specific assistance. By combining open-source tools (Node.js, PostgreSQL, Expo) with free-tier AI services (OpenRouter), the project proves that feature-rich, production-grade applications can be built cost-effectively for emerging markets.

9. Conclusion

This report presented **Event Blinker**, a comprehensive real-time event discovery platform that bridges the gap between event organizers and attendees through three key innovations: geospatial map-based event discovery, an AI-powered virtual concierge, and a peer-to-peer ride-sharing ecosystem.

The project was built on a robust, production-grade technology stack comprising:

- **Backend:** Node.js/Express.js with 11 API route modules (3,500+ lines of backend logic).
- **Database:** PostgreSQL with PostGIS extension, 12+ tables, 14+ spatial/performance indexes.
- **Mobile App:** React Native (Expo) with 10 screens and Zustand state management.
- **Web Portal:** React.js/Vite with 7 pages for organizer management.
- **Real-Time:** Socket.io with 10+ event types for chat and ride coordination.
- **AI:** LLAMA 3.3 via OpenRouter with 5-model fallback for high availability.

The project has successfully demonstrated:

1. **Real-Time Geospatial Discovery:** Efficient map-based event browsing using PostGIS spatial queries with sub-200ms response times and GiST indexing.

2. **Context-Aware AI Assistance: Blinker AI** provides event-specific, culturally localized responses by injecting live database context into LLM prompts, with multi-model fallback ensuring >95% uptime.

3. **Integrated Ride-Sharing:** A novel feature combining Haversine distance calculation, NPR dynamic pricing, 3-step verified driver registration, and Socket.io-based real-time ride coordination.

4. **Live Social Engagement:** Socket.io-powered real-time chat with AI bot integration, event likes, and attendance check-ins.

5. **Comprehensive Admin Control:** Multi-layer verification system for organizers, events, riders, and driver licenses through both API and standalone admin portal.

Key challenges, including PostGIS query optimization, Socket.io cross-platform compatibility, AI hallucination mitigation, database schema evolution, and mobile state persistence, were addressed through iterative design, careful indexing, and robust fallback strategies.

Looking ahead, the platform will be enhanced with in-app payment integration (Khalti/Esewa), real-time GPS ride tracking, AI-powered event recommendations, push notifications, QR-code ticketing, and an iOS release. **Event Blinker** establishes a strong, scalable foundation for modern event-driven social networking in Nepal, demonstrating that locally-built technology can deliver world-class user experiences.